

Princípios Básicos do DDD

1. **Domínio como Foco Principal:**
 - O domínio é o "mundo" do problema que o software resolve.
 - O objetivo é modelar o software para representar os conceitos e regras do domínio real.
2. **Linguagem Ubiqua (Ubiquitous Language):**
 - Uma linguagem comum e compartilhada entre desenvolvedores e especialistas de domínio.
 - Deve ser usada consistentemente no código, documentação e conversas para evitar ambiguidades.
3. **Modelagem Rica:**
 - Modelos de domínio devem capturar de forma explícita os comportamentos e as regras de negócios.

Camadas do DDD

1. **Camada de Domínio (Core):**
 - Contém as **Entidades, Objetos de Valor, Serviços de Domínio e Repositórios**.
 - Foca na lógica de negócios.
2. **Camada de Aplicação:**
 - Orquestra as operações do domínio, mas não contém lógica de negócios.
 - Serve como uma "ponte" entre o domínio e o mundo exterior.
3. **Camada de Infraestrutura:**
 - Implementa detalhes técnicos, como persistência (banco de dados) e integrações.
 - Dá suporte às camadas superiores.
4. **Camada de Apresentação:**
 - Exibe as informações e coleta entradas do usuário.
 - Pode incluir APIs, interfaces gráficas, etc.

Principais Conceitos do DDD

1. Entidades

- Objetos com **identidade única**.
- Exemplo: Pedido (*Order*), Cliente (*Customer*).

2. Objetos de Valor (*Value Objects*)

- Objetos imutáveis que não possuem identidade.
- Exemplo: Endereço (*Address*), CPF.

3. Repositórios

- Abstraem o acesso a dados, permitindo interagir com as Entidades.
- Exemplo: *OrderRepository*.

4. Serviços de Domínio

- Realizam operações que não pertencem diretamente a uma Entidade ou Objeto de Valor.
- Exemplo: Serviço para calcular frete.

5. Agregados

- Um conjunto de Entidades e Objetos de Valor que são tratados como uma única unidade de consistência.
- Exemplo: `Order` (Pedido) como um agregado que contém itens (`OrderItem`).

6. Fábricas

- Responsáveis por criar instâncias de objetos complexos.
- Exemplo: Fábrica para criar uma nova Entidade `Order` com vários itens.

7. Eventos de Domínio

- Representam algo significativo que aconteceu no domínio.
- Exemplo: `PedidoCriado`, `PagamentoRealizado`.

Quando Usar DDD?

1. **Domínios Complexos:**
 - Quando as regras de negócios são complicadas e precisam ser bem representadas no código.
2. **Colaboração Intensa com Especialistas de Domínio:**
 - Quando é essencial que o software reflita de forma precisa os processos do domínio.
3. **Evolução Contínua:**
 - Quando o domínio muda frequentemente e precisa de um modelo flexível e claro.

Benefícios do DDD

- **Alinhamento com o Negócio:** O software reflete fielmente as regras e processos do domínio.
- **Reutilização e Clareza:** Conceitos bem modelados são reutilizáveis e mais fáceis de entender.
- **Facilidade de Manutenção:** Modificar o modelo é mais simples porque ele é baseado na lógica de negócios, não em detalhes técnicos.

Desafios do DDD

- **Curva de Aprendizado:** Exige compreensão profunda dos conceitos e práticas.
- **Custo Inicial:** Modelar bem o domínio pode levar tempo.
- **Nem Sempre Necessário:** Em sistemas simples, DDD pode adicionar complexidade desnecessária.

Resumo

O **DDD** é ideal para sistemas complexos e com muitas regras de negócios. Ele ajuda a alinhar o software ao domínio real por meio de uma modelagem rica, conceitos claros e uma linguagem comum entre desenvolvedores e especialistas de domínio.

Se precisar de exemplos práticos ou uma introdução detalhada sobre cada componente, é só pedir! □

Exemplo Prático de DDD

Vamos criar um sistema de **Gestão de Pedidos** (Order Management) com **DDD**, incluindo os principais conceitos: **Entidades**, **Objetos de Valor**, **Agregados**, **Repositórios**, **Serviços de Domínio** e **Eventos de Domínio**.

Passo a Passo

1. Entidades

Vamos criar a entidade `Order`, que representa um pedido. Ela terá uma identidade única (`Id`) e itens.

```
public class Order
{
    public Guid Id { get; private set; }
    public DateTime CreatedAt { get; private set; }
    public List<OrderItem> Items { get; private set; }
    public decimal Total { get; private set; }

    public Order()
    {
        Id = Guid.NewGuid();
        CreatedAt = DateTime.UtcNow;
        Items = new List<OrderItem>();
    }

    public void AddItem(OrderItem item)
    {
        if (item == null) throw new ArgumentNullException(nameof(item));
        Items.Add(item);
        CalculateTotal();
    }

    private void CalculateTotal()
    {
        Total = Items.Sum(i => i.Total);
    }
}
```

2. Objeto de Valor

O item do pedido (`OrderItem`) é um **Value Object** porque sua identidade é irrelevante no domínio. O que importa são os valores.

```
public class OrderItem
{
    public string ProductName { get; private set; }
    public int Quantity { get; private set; }
    public decimal UnitPrice { get; private set; }
    public decimal Total => Quantity * UnitPrice;

    public OrderItem(string productName, int quantity, decimal unitPrice)
    {
        if (string.IsNullOrEmpty(productName)) throw new
ArgumentException("Product name is required.");
        if (quantity <= 0) throw new ArgumentException("Quantity must be
greater than zero.");
        if (unitPrice <= 0) throw new ArgumentException("Unit price must
be greater than zero.");

        ProductName = productName;
        Quantity = quantity;
        UnitPrice = unitPrice;
    }
}
```

3. Repositório

O **Repositório** encapsula o acesso ao banco de dados. Vamos criar um repositório para pedidos.

```
public interface IOrderRepository
{
    void Add(Order order);
    Order GetById(Guid id);
    IEnumerable<Order> GetAll();
}
```

Implementação usando uma lista em memória (simulação):

```
public class InMemoryOrderRepository : IOrderRepository
{
    private readonly List<Order> _orders = new List<Order>();

    public void Add(Order order)
    {
        _orders.Add(order);
    }

    public Order GetById(Guid id)
    {
        return _orders.FirstOrDefault(o => o.Id == id);
    }

    public IEnumerable<Order> GetAll()
    {
        return _orders;
    }
}
```

4. Serviço de Domínio

Vamos criar um serviço de domínio para realizar operações específicas, como criar um pedido e calcular o total.

```
public class OrderService
{
    private readonly IOrderRepository _repository;

    public OrderService(IOrderRepository repository)
    {
        _repository = repository;
    }

    public Order CreateOrder(List<OrderItem> items)
    {
        var order = new Order();

        foreach (var item in items)
        {
            order.AddItem(item);
        }

        _repository.Add(order);

        // Publicar evento (simulação)
        Console.WriteLine($"Order Created: {order.Id}, Total:
{order.Total:C}");

        return order;
    }
}
```

5. Evento de Domínio

Um evento de domínio pode ser disparado após a criação de um pedido.

```
public class OrderCreatedEvent
{
    public Guid OrderId { get; }
    public decimal Total { get; }

    public OrderCreatedEvent(Guid orderId, decimal total)
    {
        OrderId = orderId;
        Total = total;
    }
}

// Simulação de um "publicador de eventos"
public static class EventPublisher
{
    public static void Publish(OrderCreatedEvent orderEvent)
    {
        Console.WriteLine($"Event Published: OrderId =
{orderEvent.OrderId}, Total = {orderEvent.Total:C}");
    }
}
```


6. Uso do Sistema

```
class Program
{
    static void Main(string[] args)
    {
        // Repositório (In-Memory)
        IOrderRepository repository = new InMemoryOrderRepository();

        // Serviço de domínio
        var orderService = new OrderService(repository);

        // Criar itens do pedido
        var items = new List<OrderItem>
        {
            new OrderItem("Laptop", 1, 1500.00m),
            new OrderItem("Mouse", 2, 25.00m)
        };

        // Criar pedido
        var order = orderService.CreateOrder(items);

        // Publicar evento de domínio
        var orderEvent = new OrderCreatedEvent(order.Id, order.Total);
        EventPublisher.Publish(orderEvent);

        // Recuperar pedido pelo repositório
        var savedOrder = repository.GetById(order.Id);
        Console.WriteLine($"Saved Order: {savedOrder.Id}, Total:
{savedOrder.Total:C}");
    }
}
```

Saída Esperada

```
Order Created: 123e4567-e89b-12d3-a456-426614174000, Total: $1,550.00  
Event Published: OrderId = 123e4567-e89b-12d3-a456-426614174000, Total:  
$1,550.00  
Saved Order: 123e4567-e89b-12d3-a456-426614174000, Total: $1,550.00
```

Conclusão

Esse exemplo prático demonstra como usar os conceitos de **DDD** para modelar um sistema de **Gestão de Pedidos**, separando claramente as responsabilidades:

1. **Entidades:** `Order`.
2. **Objetos de Valor:** `OrderItem`.
3. **Repositório:** `IOrderRepository`.
4. **Serviço de Domínio:** `OrderService`.
5. **Eventos de Domínio:** `OrderCreatedEvent`.

Se precisar de mais detalhes ou ajuda para expandir o exemplo, é só pedir! ☐